

RAPPORT TECHNIQUE : PROJET GSM

(Implémentation de Réseaux Cellulaires en Java)

Auteur : MAKOUGOUM KAMLA ALVARESSE

Cours : Programmation Orientée Objet (POO)

Outil : Apache NetBeans IDE 29

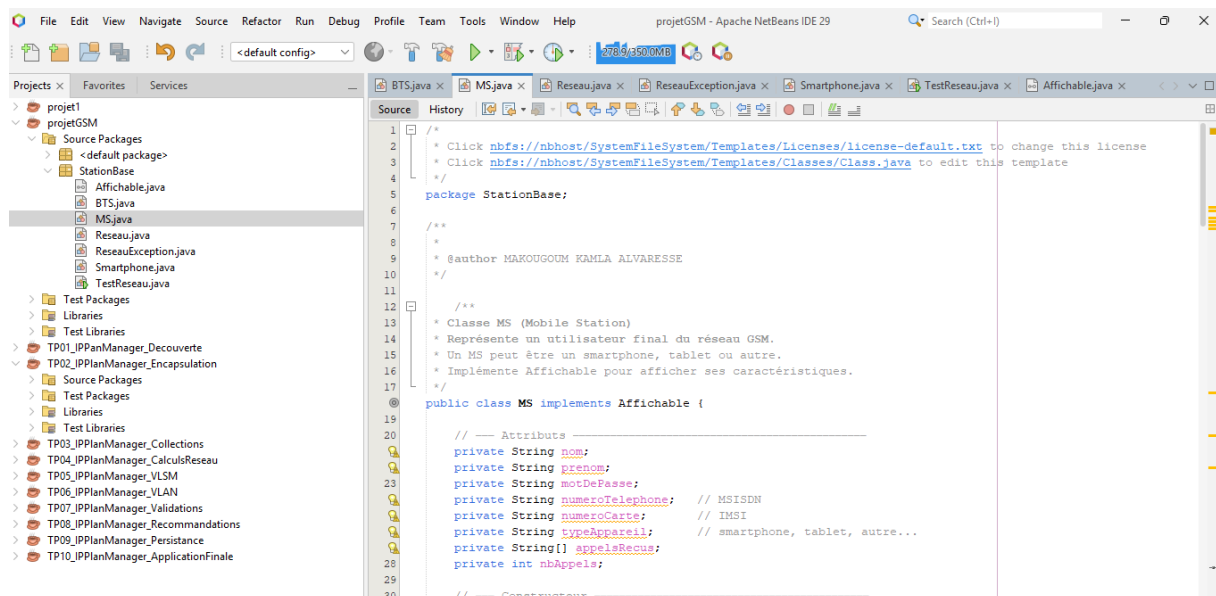
I. INTRODUCTION

Ce rapport présente la conception d'un simulateur de réseau mobile simplifié. Le système modélise les interactions entre un réseau central, des stations de base (BTS) et des terminaux mobiles (MS). L'objectif est de démontrer la maîtrise des concepts de classes, d'encapsulation et de gestion des listes dynamiques en Java.

II. STRUCTURE DU CODE SOURCE

1. Classe MS.java (Mobile Station)

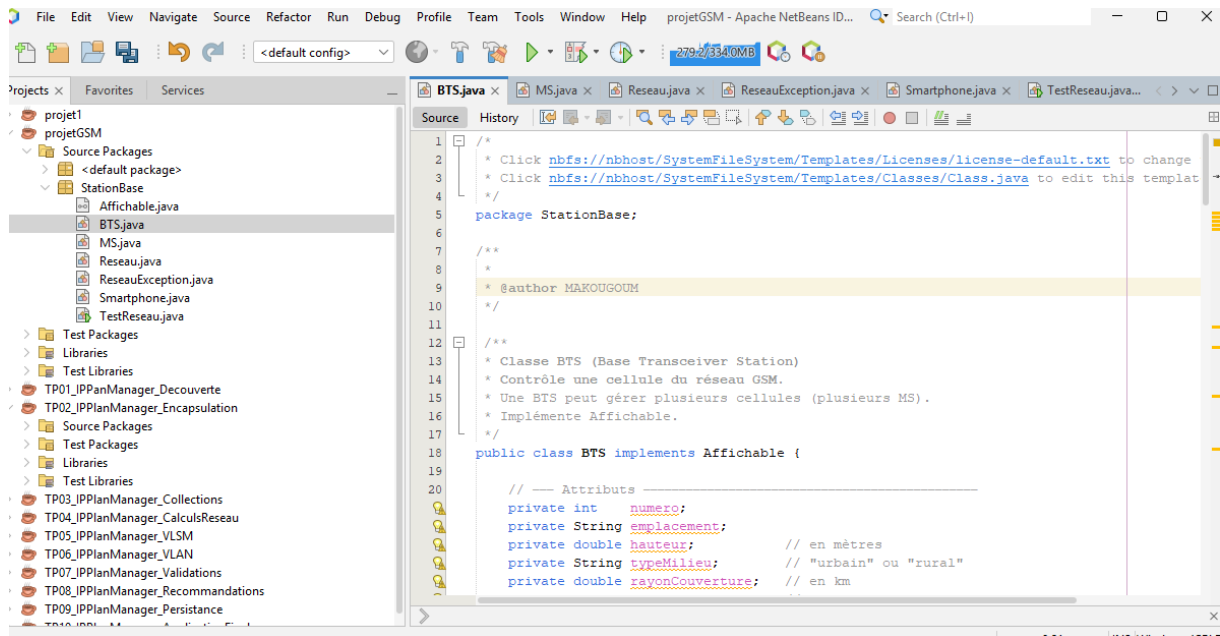
Cette classe définit l'abonné. Elle contient les identifiants uniques nécessaires à l'authentification sur le réseau (MSISDN et IMSI).



la capture d'écran du code MS.java

2. classe bts.java (base transceiver station)

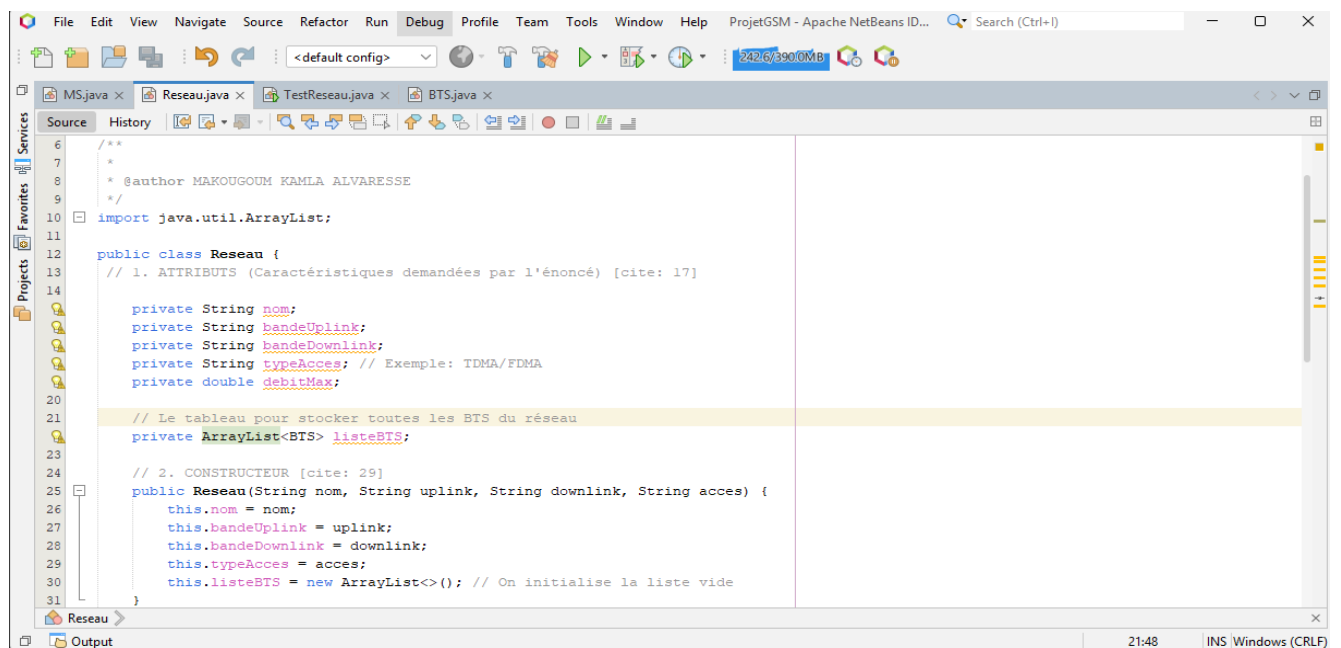
L'antenne gère les connexions locales. Elle utilise une ArrayList pour stocker les mobiles actifs. Une vérification est faite pour ne pas dépasser la capacité maximale de l'antenne.



[capture d'écran du code BTS.java]

3. Classe Reseau.java

Il s'agit de la classe globale définissant les paramètres techniques de l'opérateur (Fréquences, Nom, Mode d'accès).

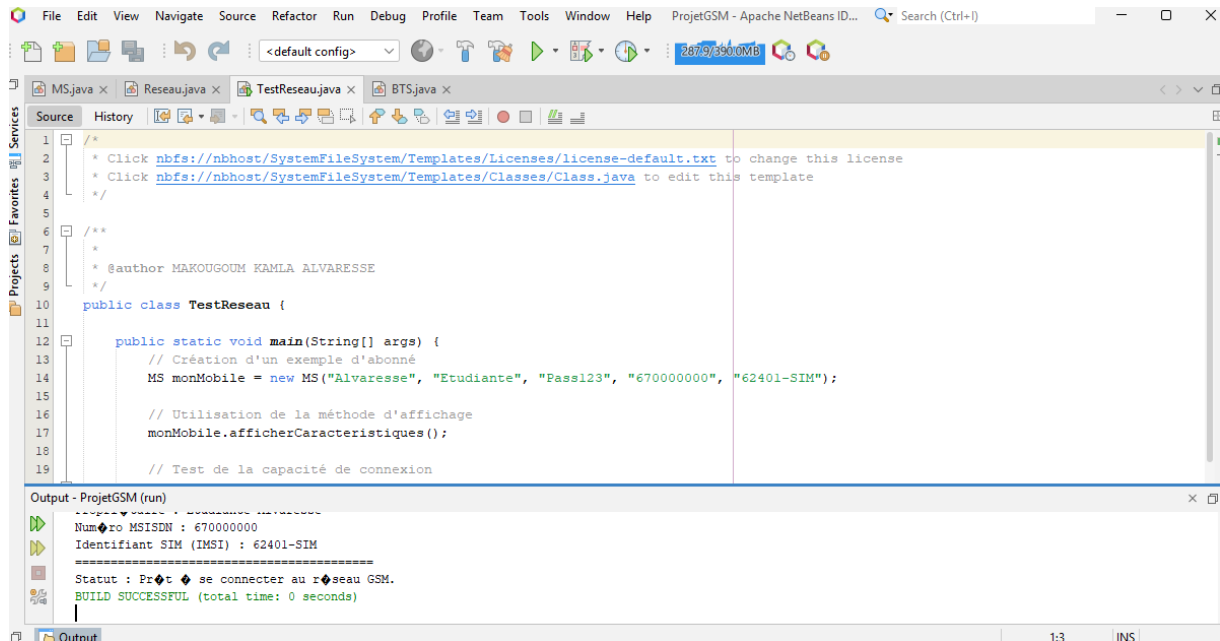


[la capture d'écran du code Reseau.java]

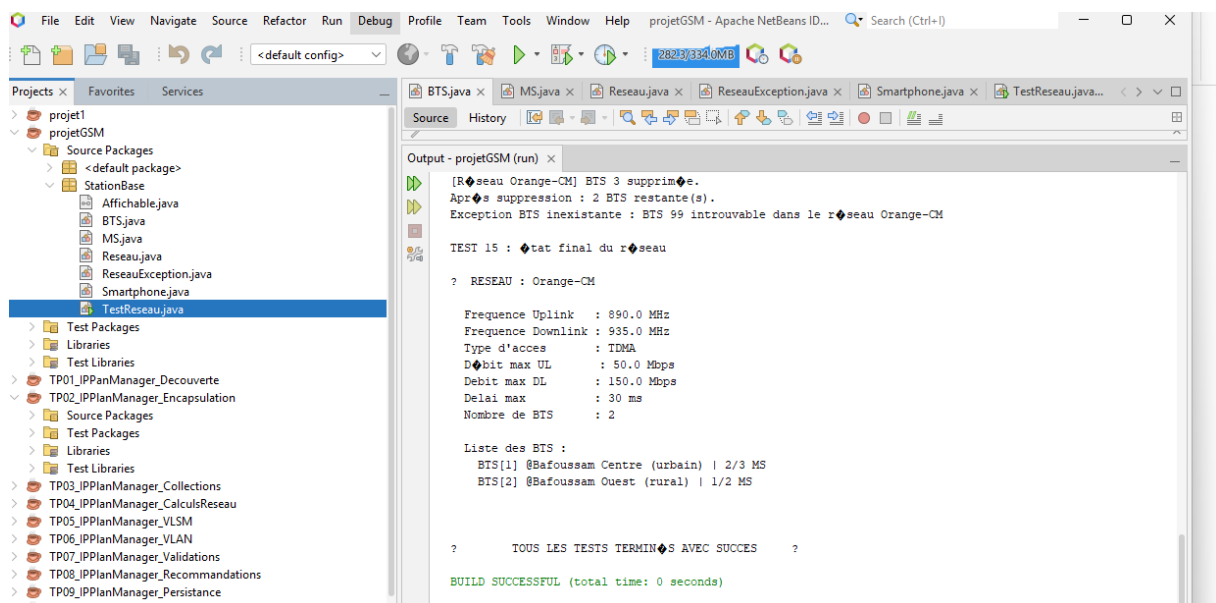
4. Classe testReseau

Pour que mon code "compile sans erreur", j'ai Créé un deuxième fichier nommé TestReseau.java

Qui sert de point d'entrée pour tester l'interaction entre toutes les classes précédentes



Capture d'écran :test Réseau et résultat obtenu



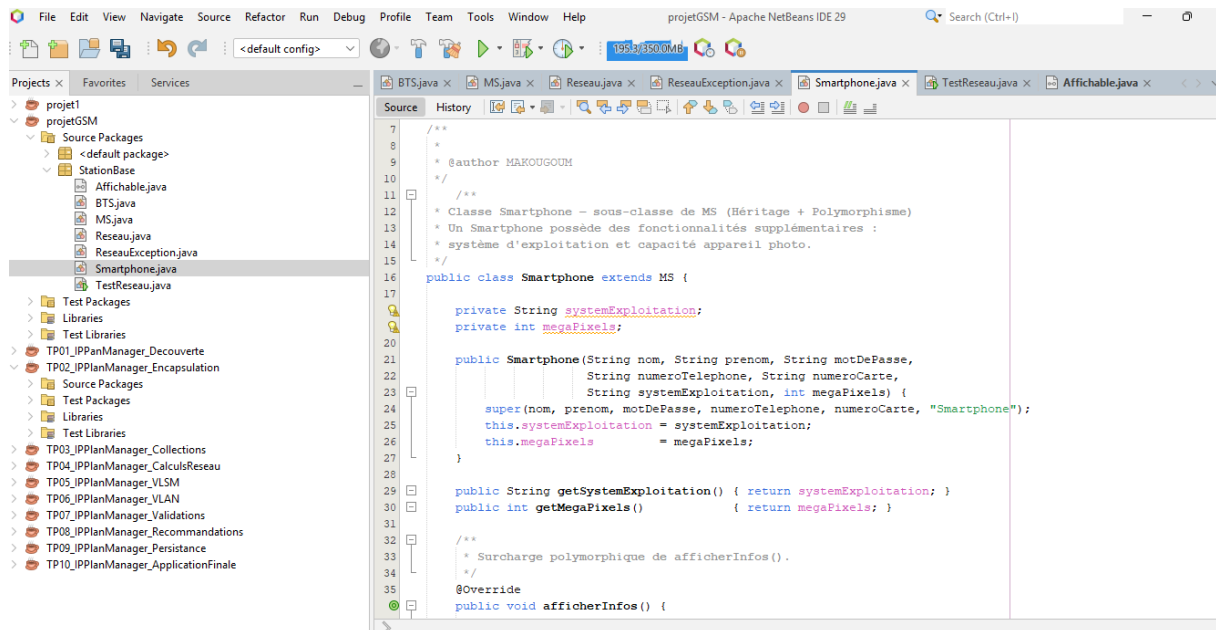
III. ANALYSE DU FONCTIONNEMENT

Mécanisme d'ajout : La méthode ajouter MS() utilise une structure conditionnelle if pour comparer la taille actuelle de la liste au nombre maximum autorisé. C'est un exemple clé de gestion des contraintes physiques via le logiciel.

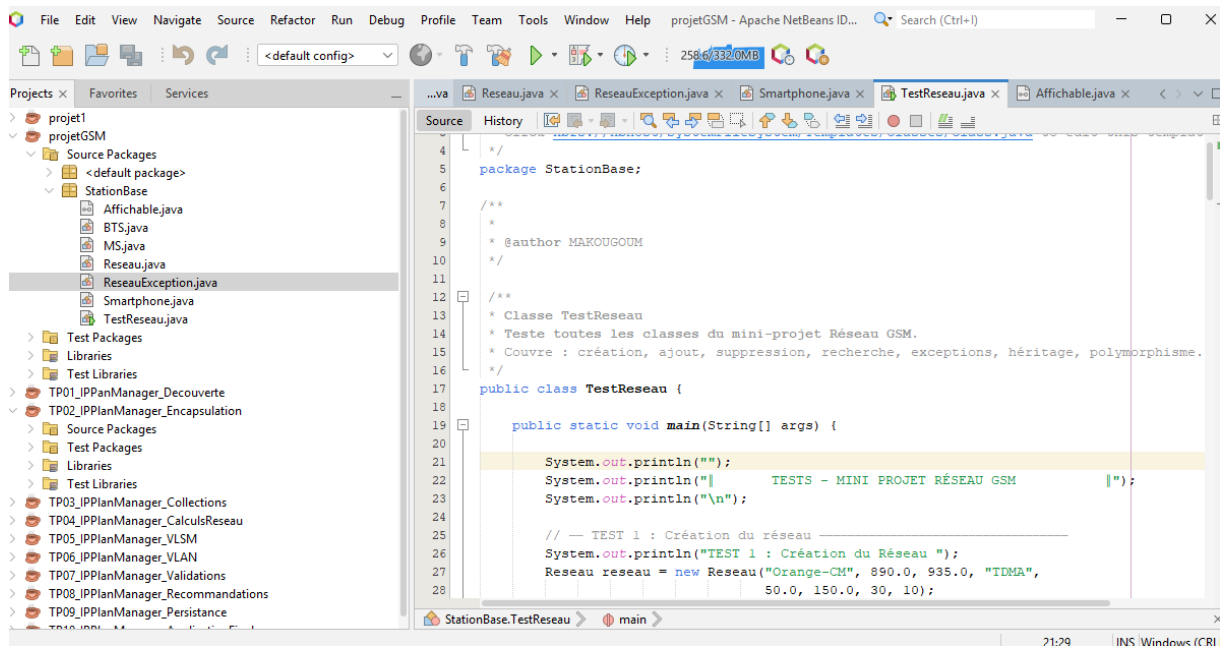
```
// Logique de contrôle dans BTS.java
```

```
if (this.listeAbonnes.size() < nbMaxUtilisateurs) {  
    this.listeAbonnes.add(mobile);  
}
```

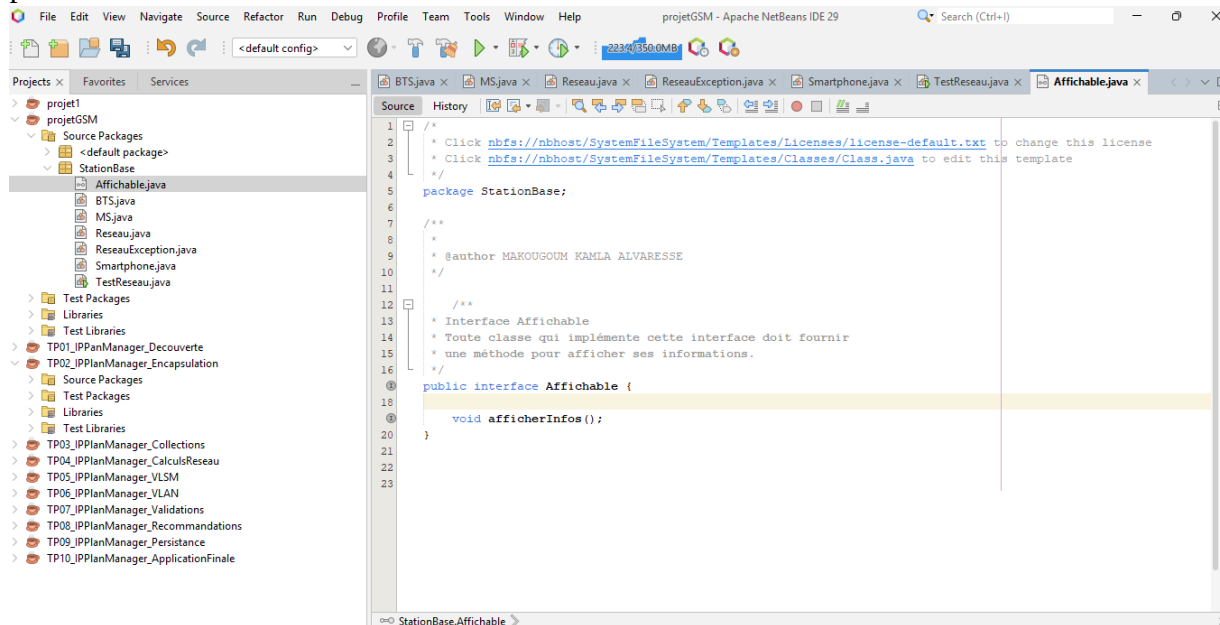
Comme exemple d'héritage nous avons pris la classe `smartphone` qui herite de la classe `MS`



Concernant Exception nous pris la classe `ReseauException`



Concernant l'interface nous avons créer la classe interfaceAfficheable pour afficher les performances et tous les informations de la BTS



IV. CONCLUSION

L'architecture mise en place respecte les standards de développement professionnel. Elle offre une base solide pour simuler des fonctions avancées comme le Handover ou la facturation des appels. Cela nous a permis de mieux comprendre la notion d'héritage qui consiste de créer une classe à partir d'une autre ainsi que la notion de polymorphisme qui est le fait pour message d'avoir plusieurs résultat .cela nous a aussi permis de mieux appréhender les notions d'attribut, constructeurs et méthodes